

ORB Feature Extraction and Matching in Hardware

Josh Weberruss*, Lindsay Kleeman, Tom Drummond

Department of Electrical and Computer Systems Engineering

Monash University, Australia

{firstname.lastname}@monash.edu

Abstract

The ORB (Oriented FAST (Features from Accelerated Segment Test) and Rotated BRIEF (Binary Robust Independent Elementary Features)) feature extractor is the state of the art in wide baseline matching with sparse image features for robotic vision. All previous implementations have employed general-purpose computing hardware, such as CPUs and GPUs. This work seeks to investigate the applicability of special-purpose computing hardware, in the form of Field-Programmable Gate Arrays (FPGAs), to the acceleration of this problem. FPGAs offer lower power consumption and higher frame rates than general hardware. A working implementation on an Altera Cyclone II (a low-cost FPGA suitable for development work, and available with a camera and screen interface) is described.

1 Introduction

A recurrent and crucial problem in robotics is the localisation of a robotic vehicle in space. Accurate positional information is vital for ensuring a robot does not get lost, and for ensuring that it is able to perform its task.

Visual odometry is a family of algorithms that are used to track the motion of a robotic vehicle in space, using only information from one or more cameras. These algorithms provide a piecewise path representing the movement of the robot and the environment around it, allowing for localisation of the vehicle. There are several different approaches to visual odometry, the most common of which track sparse (point-based) features through multiple frames of video.

Many implementations of visual odometry have already been demonstrated over the preceding decades,

but most previous efforts have focused on CPU implementation. This paper examines the applicability of dedicated hardware to the first part of the visual odometry problem, namely the identification of sparse feature correspondences between images, in the context of a Field-Programmable Gate Array (FPGA) processor.

FPGAs are large-scale integrated circuits that are designed to be reprogrammed in the field by means of switching many tens of thousands of internal logic elements. They are useful in the prototyping of application-specific integrated circuits (ASICs), because they do not require tooling up a semiconductor foundry for each minor design revision, at the cost of maximum clock speed and some extra power usage.

Our system has successfully performed feature detection and brute-force matching at 19.2 MPix/s on development board hardware, the fastest rate the camera could feed pixels to the hardware. Due to the vertical blanking interval, in which time the camera sensor collects light from the image, the logic is sitting idle for approximately 2/3 of the time, so at present, performance is entirely limited by the camera.

1.1 Problem Specification

The problem is specified as follows. The hardware system receives at its input one pixel from an input image frame per clock cycle. The pixels are received in a row-major raster scan from the camera, with a vertical blanking interval between frames. Each new frame of video is marked (outside the scope of this paper) as being a keyframe or a regular frame. Keyframes are considered reference frames — all the features in such a frame are indexed and used as a comparison reference for following regular frames. The output of the system should be, for each regular frame, a list of point correspondences. Each correspondence is a pixel coordinate in the regular frame paired with a matching pixel coordinate in the reference frame.

Thanks to algorithms of the RANSAC family, the output of the system is allowed to generate a small number of false positive matches (outliers). Repeating pat-

*This work was supported by the Australian Research Council Centre of Excellence for Robotic Vision (project number CE140100016)

terns in the input images can cause such problems, where matches can be ambiguous.

2 Hardware Platform

The current implementation platform for this project is a Terasic DE2-70 development board, which features an Altera EP2C70F FPGA. The board is connected to a Terasic TRDB-D5M camera by a 40-pin ribbon cable, and to a TRDB-LTM 800 × 480 resistive touch LCD screen by another 40-pin ribbon cable.

The DE2-70 has a number of switches, LEDs and hex displays available for input and diagnostic output, as well as two 32 MiB¹ SDRAM modules, a 2 MiB SSRAM module and an 8 MiB flash memory module.

The EP2C70F is from Altera’s Cyclone II family, which is an older architecture, but offers an adequate level of performance for the preliminary stages of the project. It has 68000 logic units, each of which has a 4-input LUT and a 1-bit D flip-flop (*Cyclone II Architecture* 2007); 250 M4K on-chip SSRAM modules with space for 4 kbit plus an optional 1/8 parity; and 300 9-bit hardware multipliers.

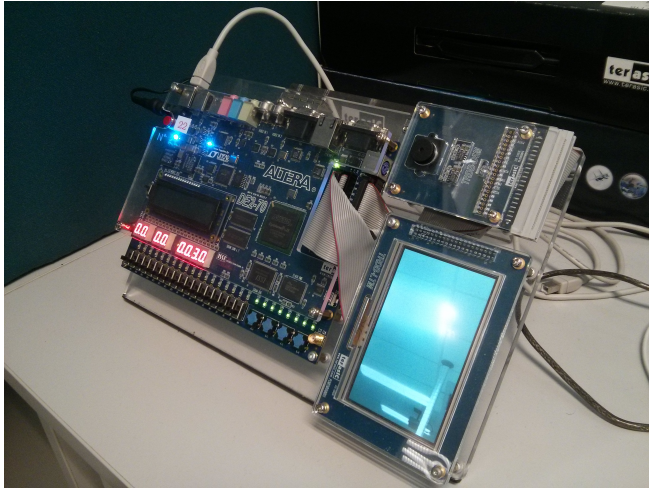


Figure 1: The Terasic DE2-70 FPGA development board used for testing

FPGAs consist of a large number of logic cells, each of which follows a structure similar to that in figure 2. The cell has a number of general inputs, a carry in, to allow for carry propagation from adjacent blocks, and a clock input for synchronous operation. The inputs are connected to the rest of the cell through a programmable lookup table (LUT). A logic cell typically contains a full adder and a D-flip flop. These logic cells are arranged with routing logic surrounding them, allowing

them to be connected in a multitude of different arrangements to suit the design being implemented. In addition, most commercial FPGA units include dedicated hardware implementations of functions like multiplication, phase-locked loops for clock multiplication, and blocks of static RAM for storing data on-chip.

FPGAs are programmed by loading values into hundreds of thousands of D-flip flops, which control multiplexers to select the functions of the LUTs and to select the paths followed by signals through the routing logic. FPGAs are typically volatile devices, in that they lose their programming when power is cut, so most practical devices load the programming from an off-chip flash memory start on bootup.

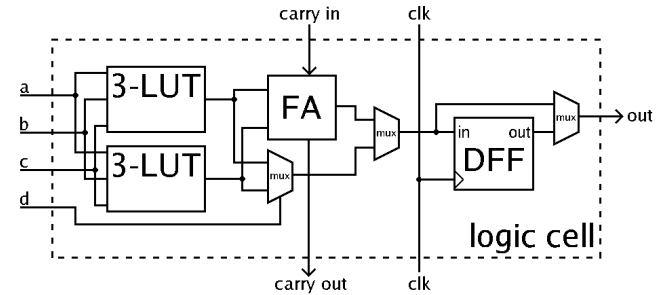


Figure 2: An example of the layout of an FPGA logic cell, from https://commons.wikimedia.org/wiki/File:FPGA_cell_example.png

The hardware configuration used on the FPGA is derived from a sample project provided by Terasic designed to demonstrate usage of the D5M camera and LTM screen. The original project simply streams pixels from the camera to a framebuffer on the SDRAM, displaying them using a separately-clocked system which streams pixels from the framebuffer to the LTM display. The code provided is written in the Verilog-2001 Hardware Description Language (HDL).

Our additions to the system are written in SystemVerilog, a language that offers significant improvements to usability when dealing with the multidimensional arrays that appear frequently in computer vision algorithms. The code inserts extra stages into the data stream from the camera to the framebuffer, allowing additional processing and manipulation of the pixel stream.

Development of the SystemVerilog code is undertaken using Quartus II 13.0.1 SP1, which is the last version of the software to support the Cyclone II architecture. The Quartus IDE provides a compiler, fitter and timing analysis package, and is able to call out to ModelSim to perform simulations of the design.

We have also written a number of testbench programs to accompany the synthesisable HDL, whose purpose is to test the accuracy of the code modules by compar-

¹1 Megabyte (MiB) = 2²⁰ bytes

ing the outputs for given inputs against reference data. For non-trivial reference data, several accompanying programs have been written in the Rust programming language², which calculate test output using CPU-based algorithms.

3 Components Used

This section describes the components shown in 3 in detail.

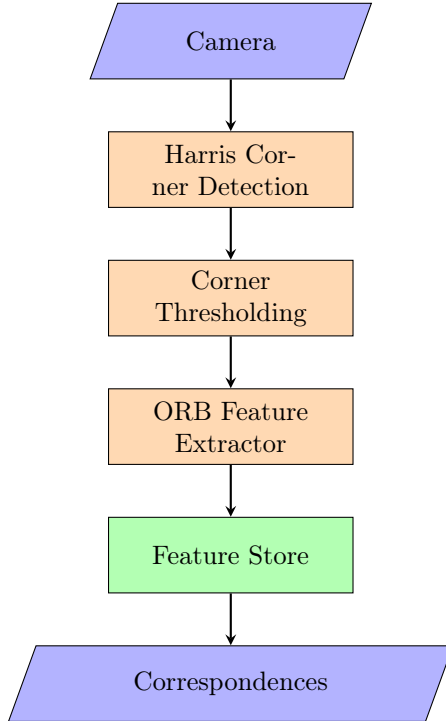


Figure 3: An overview of the data flow in the system. Blue blocks are I/O, orange blocks are processing and green blocks are storage.

3.1 Harris-Stephens Corner Detection

The Harris corner metric was introduced in (Harris and Stephens 1988) as a general corner detector. Calculation of the metric takes approximately 18 times as long as FAST (see section 3.3) when implemented on CPU (Rosten and Drummond 2005), but in general provides better-quality features. The calculation looks at the eigenvalues of a Gaussian-filtered Harris matrix to determine a corner score. The terms of the Harris matrix are the outer product of the Sobel operator applied in the x and y directions at the test point.

²www.rust-lang.org

$$H(x, y) = \begin{bmatrix} S_x^2 & S_x \times S_y \\ S_y \times S_x & S_y^2 \end{bmatrix}$$

$$H'(x, y) = H(x, y) * G_k$$

$$R(x, y) = \det(H') - k \text{Tr}(H')$$

where $H(x, y)$ is the Harris matrix, G_k is a discrete Gaussian filter kernel, k is a small ($0.04 < k < 0.15$) constant, and R is the cornerness score of the point. It is critical to apply the Gaussian smoothing kernel, because the H matrix is always singular by construction. Harris gives a numerical score, rather than a binary yes/no, allowing corner proposals to be sorted based on quality.

A further non-maximum suppression pass is usually applied to Harris corners after the scores are calculated. This examines the 8 pixels surrounding the test pixel, and only reports the pixel as passing the test if its score is higher than those of the pixels around it. This ensures that only one pixel is chosen for each corner.

In general, the Harris algorithm provides more robust corners than FAST, which are less sensitive to noise, and thus provide better results. The cost of using Harris over FAST is in computational complexity, meaning that it is generally not used as the first line feature detector in CPU implementations, but instead only used as a second pass to select high quality corners from the candidates returned by FAST.

3.2 Corner Thresholding

The Harris-Stephens corner detector returns a numerical corner score for each corner it returns. This score represents the algorithm’s confidence that the given point is in fact a corner, as opposed to a point along an edge or a point in the middle of a flat colour area. The corner score will be higher for corners with stronger contrast, for instance between white and black, than for corners with little contrast, such as between two different shades of grey.

We can apply a threshold to these corner scores in order to filter out low-quality corners and only retain points that are corners with high confidence. The choice of the threshold is a tunable parameter of the system, with the exact value dependent on the contrast of the image as a whole and the number of corners desired. If the image is generally high-contrast, it may be necessary to use a higher threshold value to ensure that the number of output corners does not swamp the feature store.

3.3 ORB Feature Extractor

Oriented FAST and Rotated BRIEF (ORB) was introduced in (Rublee et al. 2011) as a rotation-invariant development of BRIEF (Calonder et al. 2010). It achieves this invariance by rotating the BRIEF sample mask according to the orientation of the corner.

In the original paper, the feature detector FAST(Rosten and Drummond 2005) (Features from Accelerated Segment Test) is always used, hence its mention in the name. FAST is in general a less robust feature extractor than Harris-Stephens, because it considers fewer pixels. The ORB paper specifically requires using Harris-Stephens corner scores to refine the set of candidate corners returned by FAST. Since this hardware implementation calculates the Harris-Stephens score for every pixel anyway, there is no advantage to calculating FAST corners, and instead Harris-Stephens corners are used exclusively.

The orientation of the corner is determined using the intensity centroid of the image patch surrounding the point, using the following formulation (after Rosin 1999):

$$m_{pq} = \sum_{x,y} x^p y^q I(x,y)$$

$$C = \left(\frac{m_{10}}{m_{00}}, \frac{m_{01}}{m_{00}} \right)$$

The angle to the centroid from the centre of the patch is given by $\text{atan2}\left(\frac{m_{10}}{m_{00}}, \frac{m_{01}}{m_{00}}\right) = \text{atan2}(m_{01}, m_{10})$, so it is unnecessary to perform the division.

This orientation is used to transform the samples used in the BRIEF tests. ORB uses a modified set of binary tests, determined through a machine learning process to be as uncorrelated as possible. Each sample point is the average of a 5×5 window of pixels around the test coordinate, and the points are taken from a 31×31 sample space.

ORB is a good feature extractor for fast corner matching, owing to its good matching performance and high speed(Hartmann, Klussendorff, and Maehle 2013). It is accordingly used in many practical SLAM systems.

3.4 Feature Store

The feature store is a database of ORB descriptors paired with coordinates from a given keyframe. Once built from the features in the keyframe, it can be searched using ORB descriptors from features in later frames to find the best match for the incoming descriptor. Storing the coordinates of the features allows numerical correspondences to be established.

3.5 Sliding Window (aka 2D Shift Register)

The sliding window is a hardware construct that enables effective access to multiple spatially-coherent pixels with extremely low latency. Many of the algorithms used in sparse computer vision perform simple arithmetic on pixels offset from a central pixel, such as FAST corners. The naïve approach to accessing these offset pixels is to store

the whole image in a random-access memory, and request each pixel one-by-one. On a standard CPU, the image would be stored in DDR3 SDRAM, with up to three layers of SRAM cache backing. Even accessing the L1 cache requires 1 clock cycle for every two pixels (on Haswell)(Fog 2014). The FAST algorithm requires accessing 17 pixels, so this is a minimum of 9 clock cycles just to read in the required pixel values, disregarding any processing time. On a custom hardware platform like an FPGA, we can take advantage of parallelism to make all of the pixels available to combinational logic, allowing calculation of FAST corners within one clock cycle.

A shift register is a structure in which a chain of registers are arranged so that the output of one feeds the input of the next. All the registers are clocked from the same source, so an input signal progresses through the chain, one step every clock cycle. This allows access to n previous samples, where n is the number of registers in the chain.

Uncompressed video signals are typically transmitted in row-major order, where each horizontal scanline is transmitted left to right, top to bottom, one pixel per clock. If a shift register is constructed so that it is multiple rows in length, we will have access to pixels adjacent in all directions. If a given pixel is transmitted at $t = t_0$, the pixel above it will be transmitted at time $t = t_0 - \text{RowLen}$, the pixel below at time $t = t_0 + \text{RowLen}$, the pixel to the right of it at $t = t_0 + 1$, and so on.

$t_0 - \text{RowLen} - 1$	$t_0 - \text{RowLen}$	$t_0 - \text{RowLen} + 1$
$t_0 - 1$	t_0	$t_0 + 1$
$t_0 + \text{RowLen} - 1$	$t_0 + \text{RowLen}$	$t_0 + \text{RowLen} + 1$

Figure 4: Arrival times for pixels in a small patch

Because all physically-realisable filters must be causal, that is to say, not dependent on future data, we must delay output until we have all the data available. In the case of FAST corners, the required window is 7×7 , with the output pixel (i.e. the pixel we are testing) located at $(3, 3)$. This requires a delay of $3 \times \text{RowLen} + 3$ clocks. For a row length of 800 and a back porch (additional clock cycles of non-data at the end of the row) of 200, this would take 1003 clock cycles, or roughly $10\mu\text{s}$ at 100 MHz.

One cost of using such a structure is high logic element usage. For 8bpp luma, 7 rows of 800 requires 44800 logic cell registers if all pixels are to be stored. The Cyclone II FPGA on the DE2-70 has only 68000 logic cells in total, so this simple shift register would use 2/3 of the total storage.

One optimisation that can be made is to employ the on-chip SRAM to store the pixels outside of our region

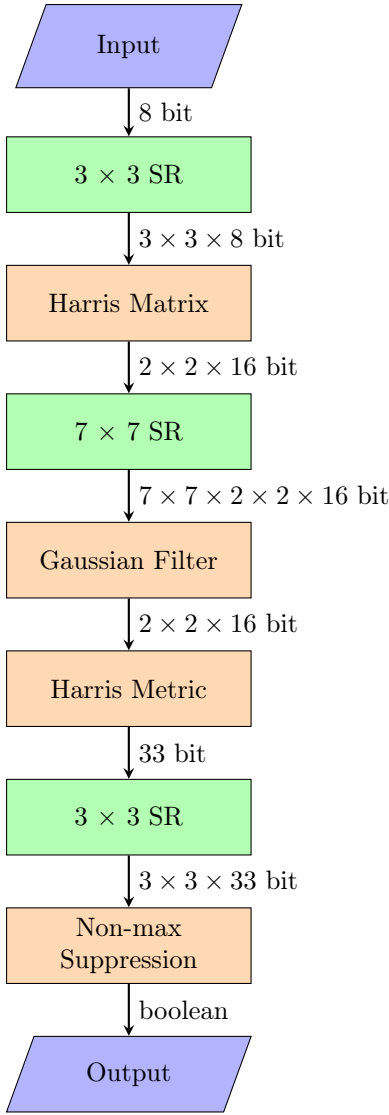


Figure 5: Data flow in the Harris-Stephens corner detector

of interest. If the dual-ported RAM is made to store one column of pixels per address (by connecting multiple memories in parallel), the read port can supply the next column of pixels, while the write port updates another column with a newly-available pixel value every clock. This cuts the logic element storage for the previous example down to 392 bits. The M4K blocks on the Cyclone II are used in $1K \times 4$ bit mode, so 12 are needed, out of the 250 present on the DE2-70 (*Cyclone II Architecture* 2007).

4 FPGA Implementation

4.1 Harris-Stephens Corner Detection

The implementation makes use of several layers of sliding windows. The first is a 3×3 window that stores

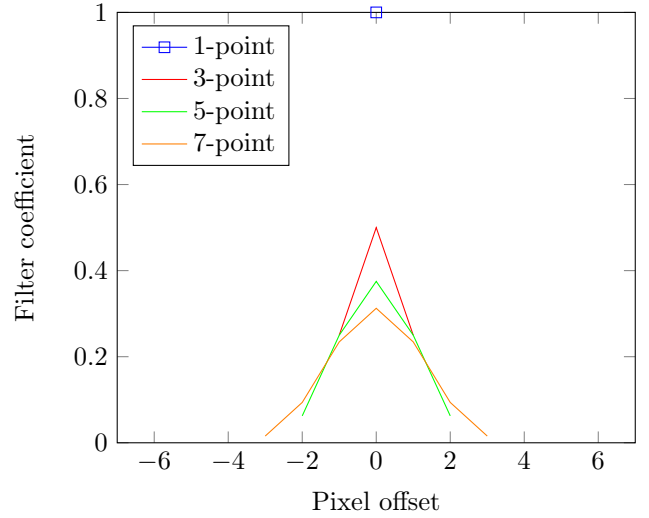


Figure 6: Pascal's triangle approximations of Gaussian filters

luma values for the Sobel operator. The outputs from these are outer-multiplied to calculate the Harris matrix. Using fixed-function hardware multipliers on the FPGA, this whole calculation takes roughly a full clock period at 100 MHz.

Each component of the Harris matrix goes into its own sliding window. This window is used to perform Gaussian blurring, in our implementation with a 7×7 filter. The multiplication coefficients are chosen as the outer products of rows of Pascals triangle, which approximates a Gaussian distribution in the limiting case (see figure 6). The filter coefficients have the favourable property that their sum is always a power of two, as each row in the triangle has double the sum of the previous row. This allows the denominator to be implemented as a right shift of the sum of products, rather than requiring a divider.

The coefficient multiplications are all constant multiplications, which can be optimised into shifts and adds automatically by the compiler. Combining all the partial sums together requires an adder tree, with time complexity $O(\log(n))$, where n is the number of elements to be added (49 in our implementation). A cascaded adder is simply too slow in this case, and is the critical path limiting the clock speed.

Once the Gaussian filter has been applied, the formula for the Harris corner metric is applied.

$$R(x, y) = \det(H') - k \text{Tr}(H')$$

The corner scores are then fed into another 3×3 sliding window, where non-maximum suppression is performed. This ensures that each pixels corner score is strictly greater than the scores of the eight surrounding pixels, and ensures that only a single local maximum is

found for each corner.

Any pixel that passes through the non-maximum suppression, and is higher than a threshold value, is determined to be a corner feature, and is displayed to the screen by means of a pink mark drawn on several successive pixels, for ease of visual observation.

4.2 ORB Feature Extractor

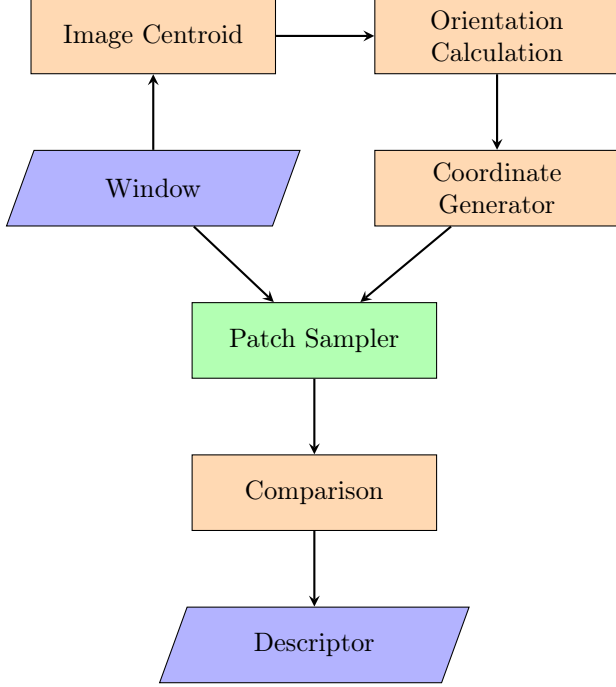


Figure 7: Data flow in the ORB feature extractor

The ORB feature extractor is implemented in logic as a pipelined process. The primary constraints on performance are multiplier availability and random-access memory read ports.

4.3 ORB Window

The ORB Window module is responsible for allowing random access to individual pixels of the ORB patch, and for calculation of the image moments for determining the patch orientation. Structurally, it closely resembles the sliding window detailed in 3.5, but with some modifications. The standard sliding window is implemented using logic cells to store the values of the patch. This is an excellent solution for filters where the samples are fixed and independent of the data. In the case where the sample coordinates can change dependent on input, such as in the ORB sampling phase, the routing logic required to enable random access is prohibitive, requiring large numbers of extremely wide and long multiplexers.

A better alternative is to make use of the M4K memory blocks once again. Each row of the ORB patch is

represented by an M4K block in a rolling fashion. A pointer is used as the read head, keeping track of the location of the next address to be written. With every incoming pixel value, the addresses of the left and right boundaries of the active patch increase by one. This allows us to randomly access into the patch window, while only requiring one write per row per clock.

This ORB Window module is also responsible for calculating the coordinates of the image centroid. We use a square windowing function for simplicity, with dimensions equal to the size of the ORB patch (i.e. 37×37). Three additional sets of values are stored - the sums of the values in each of the columns, each of the rows, and the whole patch. These values can be updated from the previous values efficiently.

Guarantees (combinational logic):

$$ColSums[x] = \sum_y Patch[y] \quad (1)$$

$$ColSums[x] = \sum_x Patch[x] \quad (2)$$

$$PatchSum = \sum_x ColSums[x] \quad (3)$$

$$= \sum_y RowSums[y] \quad (4)$$

$$m_{10} = m_x = \sum_{x,y} x Patch[x, y] \quad (5)$$

$$= \sum_x x ColSums[x] \quad (6)$$

$$m_{01} = m_y = \sum_{x,y} y Patch[x, y] \quad (7)$$

$$= \sum_y y RowSums[y] \quad (8)$$

State Transitions:

$$Patch_{t+1}[x, y] = \begin{cases} Input_t[y], & \text{if } x = 17 \\ Patch_t[x + 1, y], & \text{otherwise} \end{cases} \quad (9)$$

$$ColSums_{t+1}[x] = \begin{cases} \sum_y Input_t[y], & \text{if } x = 17 \\ ColSums_t[x + 1], & \text{otherwise} \end{cases} \quad (10)$$

$$RowSums_{t+1}[y] = RowSums_t[y] + Input_t[y] - Patch_t[-17, y] \quad (11)$$

$$PatchSum_{t+1} = PatchSum_t + \sum_y Input_t[y] - ColSums_t[-17] \quad (12)$$

$$\begin{aligned}
m_{x,t+1} - m_{x,t} &= \sum_{x=-17}^{17} x(C_{t+1}[x] - C_t[x]) \\
&= 17 \times \sum_y Input_t[y] - PatchSum_t \\
&\quad + 18 \times C_t[-17]
\end{aligned} \tag{13}$$

The various internal values are kept up to date with incoming pixel data, meaning that calculation of the image moments for image centroid computation requires no additional time. Some logic can be saved by using delta values for calculating m_x .

Since row sum calculation involves subtraction of the outgoing pixel as well as addition of the incoming pixel, we require two access ports on the memory. One is used as a write head, writing the value of the incoming pixel into the RAM, while the other is used as a read head, reading from the outgoing pixel. In random access operation mode, used when sampling the image patch to generate bits of the descriptor, both ports are used as read ports to double the throughput.

4.4 Image Centroid

The first step in the calculation of the ORB descriptor is to determine the coordinates of the image centroid. The image centroid itself is not actually calculated, as this would require a slow division operation. Instead, the two image moments m_{10} and m_{01} are output directly to the orientation calculation step, which is only concerned with the ratio between the moments.

The orientation calculation has the task of returning an approximation of $\text{atan2}(m_{01}, m_{10})$. Rublee et al. 2011 suggests using 30 different angle quantisations, corresponding to a separation of 12° , and a maximum error in angle of 6° . This number was chosen because it allowed for a reasonable number of coordinate lookup tables in the original code. The code currently shipping in OpenCV, however, uses atan2 directly, allowing full precision in the results. We elected not to implement atan2 with a CORDIC unit in hardware, but instead to use a high-resolution approximation. 64 steps yield a maximum error in angle of $2.81^\circ \left(\frac{360}{64}\right)$. Considering the patch is of radius 18 pixels, the maximum possible positional error is

$$18 \sin\left(\frac{2\pi}{2 \times 64}\right) = 0.88 \text{ pixels} \tag{14}$$

This is similar in magnitude to the quantisation error over the whole patch, and so is deemed to be acceptable.

The orientation is calculated using a lookup table. The first two steps determine the quadrant of the angle based on the signs of the two image moments. At the same time, the vector of x and y positions is rotated so that it is in the quadrant $[0, \frac{\pi}{2})$. Given the vector $[x, y]$, the

angle θ is given by

$$\tan(\theta) = \frac{y}{x}, \tag{15}$$

which can be manipulated to give

$$x \tan(\theta) = y, \tag{16}$$

requiring no divisions. Since all of our 16 possible θ values are known ahead of time, their tangents can be precalculated. Their products with x are computed, and each is tried in order, to find the angle that best matches the y value.

The coordinate generation step takes as inputs the orientation from the previous step and pairs of ORB comparison sample coordinates. The sample points are with reference to an orientation of 0° , so they must be vector rotated to align with the feature in question. Since the number of possible orientations is relatively small, the $\sin$ and $\cos$ values are precalculated and used as entries in a lookup table. Since both the sample coordinates and the rotation angle change depending on inputs, hardware multipliers must be used for the vector rotation, and not just shifts and adds. The coordinates of the samples are transformed using

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}. \tag{17}$$

Once a pair of sample coordinates has been generated, these are used as indexes into the patch image to sample pixels for comparison. The ORB Window patches are stored in M4K block memory, and considering each block has two read ports, each copy of the image patch can satisfy enough reads to generate one bit of descriptor per clock cycle.

The final step of the process is to compare the intensities of the two pixel samples returned from the patch sampler. If the first sample intensity is greater than the second, the descriptor bit is set, else it is reset. Once all 256 bits of the ORB descriptor are determined, the descriptor is returned for indexing.

4.5 Feature Store

The ORB descriptors are stored in a specialised data structure, along with their coordinates in the keyframe image. Whenever a corner descriptor is calculated by the ORB extractor, it flows through a FIFO (first-in, first-out) queue to the feature store module. If the frame being processed is a keyframe, the descriptor and its coordinates are stored in the memory of the feature store. If the frame is a regular frame, instead of being stored, the descriptor is compared against all the features currently in the store.

The comparison process is a simple brute-force linear scan, with the best match defined as the comparison that has the smallest Hamming distance. This approach was chosen because it is guaranteed to yield the best matching performance, since every possible match is checked. Moreover, considering the hardware architecture, descriptor comparisons can be performed extremely cheaply, requiring only an xor operation and a bit population count. They can also be parallelised effectively, allowing multiple comparisons to be performed per clock cycle.

4.6 Parallelism

Since the processes of generating sample coordinates and sampling are independent for each pair of samples, those steps can very easily be parallelised. Calculating one pair of coordinates and performing one pair of samples per clock would require 256 clocks to extract the whole descriptor, while having four such operations running in parallel would reduce the runtime to 64 clocks. On the development platform, each coordinate generation takes 8 9-bit DSP multipliers, and each copy of the image patch takes 4 M4K memory blocks.

To enable this usage, the ORB Window module can be instantiated multiple times for each ORB descriptor module. Each instance receives the same input pixels, meaning that all have the same contents. When a corner is detected, the input ceases, so that both ports are available for random access. Since the ORB Windows are not taking input while they are being used for calculation, any corners that occur between when the input is detached and the point when the windows have reloaded a new frame are lost. To prevent such corners being missed, the entire ORB module can be duplicated. An arbitrator manages the allocation of the ORB module resources to detected corners, selecting a free module for each corner, provided one is available.

5 Differences from OpenCV implementation

The standard reference implementation for the algorithms discussed in this paper is the OpenCV library. OpenCV includes Cuda- and OpenCV-accelerated versions of the ORB feature extractor, suitable for running on a GPU. Only a CPU version of the Harris-Stephens corner detector is offered.

5.1 Harris-Stephens Corner Detector

The reference implementation of the Harris-Stephens corner detector is designed to run on standard computer hardware, and as such operates as a multi-pass operation. The first pass calculates per-pixel corner scores and caches them, while the second pass is used to filter the corner scores. An example filtering scheme might

be to take the 200 highest scores, with the proviso that no corner can be within 3 pixels of any other previously-selected corner.

This process requires caching all the corner scores in an image and sorting them, which is inefficient for a memory-constrained FPGA device. Instead, this work takes a slightly different approach, where a hard threshold is used, and only points with a Harris-Stephens score over the threshold are accepted. The minimum distance between points is fixed at 2px, implemented with a 3×3 window non-maximum suppression filter.

5.2 ORB Feature Extractor

The primary difference between our implementation of the ORB feature extractor is in the resolution of the sample orientation process. To speed up the calculation process, the orientation is constrained to steps of $\frac{\pi}{32}$, instead of allowing the full gamut of 32-bit floating point. This can result in small misalignments of the sample mask, and yield slightly different descriptor values dependent on the rotation of the image patch.

6 Results



(a) The reference image (b) The comparison image

Figure 8: Stockholm’s Kammarrättens Hus

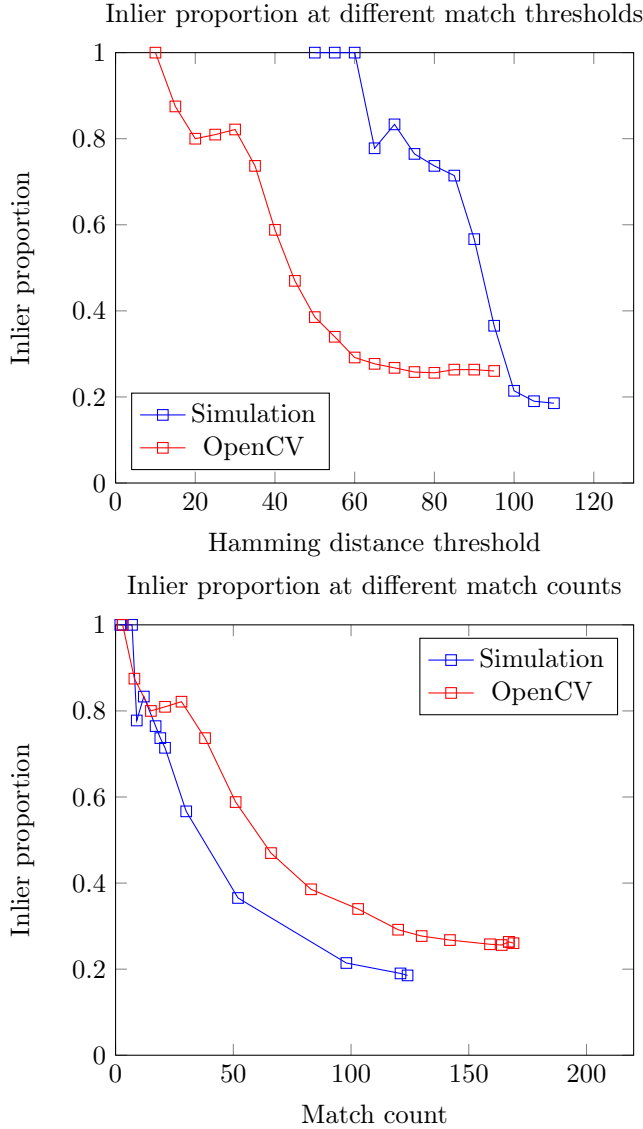


Figure 9: Comparison between simulation and OpenCV reference

Figure 9 shows how the simulated ORB implementation compares against the reference code in OpenCV on real images. The source images (figure 8) are crops of photographs of the Stockholm administrative court of appeals (Kammarrätten) building on the island of Gamla Stan from two different locations to the south. The ground truth for the image is assumed to be a simple translation of $(-32, 8)$, and inlying matches are those with an error of 8 pixels or less.

The top chart shows how the inlier proportions of the two methods change as the Hamming distance threshold is varied. The Hamming distance threshold is a controllable parameter of the system. A low threshold means that only matches that are very similar, having a small

Hamming distance between the two descriptors, are accepted, whereas a higher threshold means that more dissimilar matches are accepted.

The bottom chart shows how increases in the number of matches selected affect the proportion of matches considered inliers. The match count is a function of the threshold value selected, which is a controllable parameter. A minimum number of matches is necessary to perform RANSAC or similar processes.

As can be seen, the performance of the two implementations differs somewhat. In general, the matches detected by the simulated HDL code are of slightly lower quality, in that they are more likely to be outliers than with the reference OpenCV code. In addition, they typically have a larger Hamming distance, which is represented by the shift to the right in the top figure.

Although the performance of the HDL system is not as good as that of the OpenCV reference, with this sample image, the inlier proportion with both methods is perfectly adequate for RANSAC to yield a consensus for the image transformation.

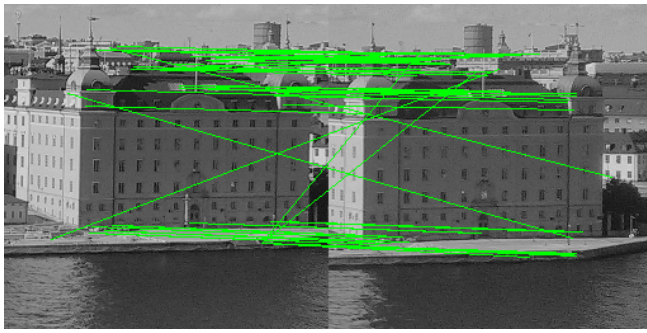
Figure 10 shows a visual representation of the difference in matching between the two methods. In each case, the threshold has been chosen to give a reasonable number of correspondences for visual inspection. Green lines connect coordinates of index points with coordinates of points in the live image (reference on left, live on right). As can be seen, the majority of the matches are parallel horizontal lines, with some outliers.

We attribute the majority of the differences between the two implementations to the approximations made in the hardware simulation, such as the higher resolution of the rotations, the differently-shaped windowing function for the centroid calculation, and the slightly different implementation of Harris-Stephens corners. If the alignment of an ORB corner is computed slightly inaccurately, some of the comparisons will be misaligned, potentially resulting in different output bits.

Hardware simulation of an HDL design presents the most effective method for development on the FPGA platform. Simulation tools provide introspection of all the data lines used in a design, allowing the developer to debug the design effectively, and to provide input and extract output without the requirement of designing FPGA/PC interfaces. Additionally, the testbench pattern, wherein known inputs are fed to a hardware design and the outputs are compared with known good values (analogous to unit testing of CPU code), can be implemented effectively with simulations.

7 Implementation

For demonstration purposes, the internal feature matching system was coupled to I/O devices present on the DE2-70 board. The onboard camera and screen are used,



(a) OpenCV version



(b) Hardware simulation

Figure 10: Feature correspondences for the two implementations

in conjunction with switches to control keyframing. At the start of each incoming frame, the state of a particular switch is checked. If the switch is pressed, that frame is considered to be a keyframe, and indexed in the Feature Store module. If not pressed, the keypoints in the frame are matched against the indexed features.

Correspondences are displayed on the screen using bright green lines. A feature that is a good quality match with an indexed keypoint in the store has a line connecting the coordinates of the points in both frames. This means that a corner in the live image is typically connected with an uninteresting point in that live image, as shown in figure 11. Parallel lines indicate sample consensus, and suggest a high likelihood that the points in question are inliers.

The design as tested has an F_{max} of 48.48 MHz, which is the frequency that the design can safely run at under worst-case conditions, at unfavourable temperatures using the slow delay model. The development board being used is however able to run safely at a 60 MHz pixel clock.

With minimal parallelism (1 ORB module, 1 ORB window per module), the feature detection and matching uses 462.49 mW (estimated by Quartus), with the bulk of the power (277 mW) used by the feature detection process.



Figure 11: The design in operation

References

- Cyclone II Architecture* (2007). Altera Corporation. URL: http://www.altera.com/literature/hb/cyc2/cyc2_cii51002.pdf.
- Harris, Chris and Mike Stephens (1988). “A combined corner and edge detector.” In: *Alvey vision conference*. Vol. 15. Manchester, UK, p. 50.
- Rosten, Edward and Tom Drummond (2005). “Fusing points and lines for high performance tracking”. In: *Computer Vision, 2005. ICCV 2005. Tenth IEEE International Conference on*. Vol. 2. IEEE, pp. 1508–1515.
- Rublee, Ethan et al. (2011). “ORB: an efficient alternative to SIFT or SURF”. In: *Computer Vision (ICCV), 2011 IEEE International Conference on*. IEEE, pp. 2564–2571.
- Calonder, Michael et al. (2010). “Brief: Binary robust independent elementary features”. In: *Computer Vision–ECCV 2010*. Springer, pp. 778–792.
- Rosin, Paul L (1999). “Measuring corner properties”. In: *Computer Vision and Image Understanding* 73.2, pp. 291–307.
- Hartmann, Jan, JH Klussendorff, and Erik Maehle (2013). “A comparison of feature descriptors for visual SLAM”. In: *Mobile Robots (ECMR), 2013 European Conference on*. IEEE, pp. 56–61.
- Fog, Agner (2014). *Instruction Tables*. URL: http://www.agner.org/optimize/instruction_tables.pdf.
- Cyclone II Architecture* (2007). Altera Corporation. URL: http://www.altera.com/literature/hb/cyc2/cyc2_cii51008.pdf.